
1. El Concepto: Borrado por Filtro

En ingeniería de datos, existen dos formas de vaciar una tabla: borrar registro por registro (ineficiente) o ejecutar una instrucción de borrado sobre toda la colección. Isar permite hacer esto último de forma atómica.

2. Implementación en el Notifier (`providers/carrito_provider.dart`)

Agregaremos este método dentro de la clase `Carrito`.

```
Future<void> vaciarCarrito() async {
  final isar = DBService.isar;

  // Ejecutamos la transacción de escritura
  await isar.writeTxn(() async {
    // Isar permite borrar todos los objetos de una colección de golpe
    await isar.carritoItems.clear();
  });

  // Notificamos a Riverpod que el estado ha cambiado (ahora estará vacío)
  ref.invalidateSelf();
}
```

3. Integración en la UI (`screens/tienda_screen.dart`)

Ubicaremos este botón en el encabezado de la sección del carrito para que sea accesible.

```
// Dentro del Column de la TiendaScreen, en la cabecera del carrito:
Padding(
  padding: const EdgeInsets.all(8.0),
  child: Row(
    mainAxisAlignment: MainAxisAlignment.spaceBetween,
    children: [
      const Text('Tu Carrito (Persistente)',
        style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold)),

      // BOTÓN DE VACIAR
      TextButton.icon(
        onPressed: () {
          // Opcional: Podrías añadir un diálogo de confirmación aquí
          ref.read(carritoProvider.notifier).vaciarCarrito();
        },
        icon: const Icon(Icons.delete_sweep, color: Colors.red),
        label: const Text('Vaciar', style: TextStyle(color: Colors.red)),
      ),
    ],
  ),
),
```

```
),  
,
```

4. Explicación Técnica

- **isar.carritoItems.clear()**: A diferencia de **delete(id)**, el método **clear()** es una operación de alta eficiencia que limpia la colección por completo. Es importante recalcar que esto **no borra los productos** del catálogo, solo los registros de la tabla **CarritoItem**.
- **Transaccionalidad**: Al estar dentro de un **writeTxn**, si por alguna razón el proceso falla, la base de datos se mantiene intacta (principio de **Atomicidad**).
- **Gestión de UI**: Al llamar a **ref.invalidateSelf()**, el **carritoTotalProvider** (que definimos antes) se recalculará automáticamente y mostrará **\$0.00** sin que hayamos escrito una sola línea de código extra para ello. Esa es la magia de los providers derivados.

5. Reto Final

"Modifiquen el método **vaciarCarrito** para que pida una confirmación mediante un **showDialog** antes de ejecutar el borrado. Si el usuario cancela, no se debe llamar al notifier."
